Build With Claude — Day 3 Guide 2

Working with existing project in claude code

Author: Manaranjan Pradhan · www.manaranjanp.com

`ffn` (github.com/pmorissette/ffn) is a financial-functions library for Python: price series in, performance statistics, drawdowns and return tables out. It prints its reports to the terminal (`display()`), writes CSV (`to_csv()`) and draws matplotlib charts. It has no HTML output.

In this demo you add one: `PerformanceStats.to_html()` and `GroupStats.to_html()`, producing a self-contained HTML page with the summary statistics and a **colour-coded monthly returns table** (green for gains, red for losses, shaded by size). The feature is small enough for one session and real enough to need every step of the Claude Code workflow: build context in an unfamiliar 3,500-line module, write the issue, reproduce with a failing test, implement, review, open a pull request, merge.

The whole thing happens in **your own fork**. Nothing is sent upstream unless you decide to later.

Time: about 90 minutes. **Prerequisites:** Claude Code installed and logged in; a GitHub account; `git` , `gh` , Python 3.10+; macOS, Linux, WSL or Windows PowerShell. Commands marked **macOS** also apply to Linux and WSL; **PowerShell** is for Windows. Slash commands and prompts are identical everywhere.

Part 1 — Your own copy of the repository

1.1 Log in to GitHub from the terminal

```
gh auth login        # GitHub.com → HTTPS → authenticate in the browser
gh auth status       # "Logged in to github.com account <you>"
gh auth setup-git    # lets git push use the same credential
```

1.2 Fork and clone in one step

```
gh repo fork pmorissette/ffn --clone --remote
cd ffn
git remote -v
```

What you should see: `origin` → `https://github.com/<you>/ffn.git` (your fork) and `upstream` → `https://github.com/pmorissette/ffn.git` (the original). The default branch, `master` , is checked out (ffn uses `master` , not `main` ; the fork keeps the name, and the commands in this guide use `master` throughout).

Why a fork. Your pull requests will target `<you>/ffn` 's `master` , so you can merge them yourself. The original project is untouched; if you later want to contribute the feature upstream, the same branch can be opened against `pmorissette/ffn` .

If you prefer the web UI: open github.com/pmorissette/ffn → **Fork** → keep the name `ffn` → then `git clone https://github.com/<you>/ffn.git && cd ffn && git remote add upstream`

`https://github.com/pmorissette/ffn.git`.

1.3 Make the fork the default for `gh`

```
gh repo set-default <you>/ffn
```

Why. On a fork, `gh pr create` and `gh issue create` otherwise offer the *upstream* repository as the target. Setting the default once keeps every later command pointed at your fork.

1.4 Install and establish the baseline, by hand

macOS

```
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[test]" ruff
python -m pytest -q -p no:cacheprovider
```

PowerShell

```
python -m venv .venv; .\.venv\Scripts\Activate.ps1
pip install -e ".[test]" ruff
python -m pytest -q -p no:cacheprovider
```

What you should see: `1 failed, 545 passed` (about 10 seconds; the install pulls matplotlib and pandas, so it takes a minute the first time). The one failure is `tests/test_core.py::test_limit_weights_preserves_input_precision_budget[rounded-input-float16]`, a tolerance-type error unrelated to reporting. Write it down: it is a *known* failure, and the model must be told so.

Why do this before starting Claude Code. A model that sees a red suite will try to fix it. The baseline separates "already broken" from "I broke it".

Part 2 — Build the project context in Claude Code

2.1 Start in the repository root

```
claude
```

`/context` shows only the system prompt and tools: nothing project-specific is loaded yet.

2.2 `/init`

```
/init
```

Claude Code surveys the repository and drafts `CLAUDE.md`. **What you should see (ffn):** the package layout (`ffn/core.py`, `data.py`, `utils.py`, `yahoo.py`), `pytest` as the test runner, the `pyproject.toml` dependencies (pandas, numpy, matplotlib, tabulate, scipy) and extras (`test`, `develop`), the `tests/data/` fixtures, and the Makefile targets (`make lint-py` runs `ruff check` and `ruff format --check`; `make test`).

2.3 Architecture briefing in plan mode

Press `Shift+Tab` until the status line reads **plan**, then:

```
Brief me on this repository as if I were joining the project today:
1. What the library does, in two sentences.
2. ffn/core.py is 3,500 lines. Map its sections: the PerformanceStats class, the GroupStats
   class, the module-level calc_* functions, the plotting helpers, the weighting/optimisation
   functions. Give line ranges.
3. The reporting surface: every method that produces output for a human (display*, to_csv, plot*)
   on both classes. For each: what data it reads from self, and how values are formatted.
4. How return_table is built and what its index and columns are.
5. How tests are organised, where the offline price data lives, how a PerformanceStats is
   constructed in the tests without network access, and what conftest.py sets up.
6. Conventions I must follow (formatting helpers, docstring style, how numeric strategy names
   are handled, lint and test commands).
Cite file:line for every claim. Change nothing.
```

What you should see: `PerformanceStats.display()`, `display_monthly_returns()`, `display_lookback_returns()`, `to_csv()`, `plot()`, `plot_histogram()`; the same on `GroupStats` plus `plot_correlation()` and `plot_scatter_matrix()`; `_stats()` returning the `_STATISTICS` table of `(key, label, format)` tuples; `_format_stat_value()`, `fmtp` / `fmtn` / `fmtpn` in `utils.py`; `return_table` as a DataFrame of years × `Jan … Dec, YTD`; `tests/data/test_data.csv` (AAPL, MSFT, C, 2004–2014) read directly in the tests (`conftest.py` only sets the matplotlib backend to Agg); the `.calc_stats()` accessor registered on pandas objects when `ffn` is imported.

Leave plan mode (`Shift+Tab` back to default).

2.4 Write `CLAUDE.md`

```
Rewrite CLAUDE.md from the briefing, under 60 lines:
- two-line description
- setup and test commands; note that
  tests/test_core.py::test_limit_weights_preserves_input_precision_budget[rounded-input-float16]
  is a KNOWN FAILURE unrelated to reporting and must not be "fixed" as part of other work
- module map of ffn/core.py with line ranges
- the reporting surface and the rule that new report methods mirror to_csv: same _stats()
  source, same formatting helpers, path=None returns a string, path given writes the file
  and returns None
- offline test data: tests/data/test_data.csv; never call ffn.get() or the network in tests
- conventions: `make lint-py` (ruff check + ruff format --check) must pass, docstrings in the
existing style,
  str(name) for strategy names because names can be numeric, tests next to the feature in tests/
```

Then:

```
/clear
/context all
```

`CLAUDE.md` is now under **Memory files**. Ask the proof question:

```
Which test is a known failure in this repository and why must it be left alone?
```

2.5 Permissions for this session

```
Create .claude/settings.local.json with:
{
  "permissions": {
    "allow": ["Read", "Grep", "Glob", "Edit",
              "Bash(python -m pytest *)", "Bash(git status *)", "Bash(git diff *)",
              "Bash(git log *)", "Bash(git add *)", "Bash(git commit *)", "Bash(git checkout -b
*)"],
    "ask":   ["Bash(git push *)", "Bash(gh pr *)", "Bash(pip *)"],
    "deny":  ["Read(./.env)", "Bash(git push --force *)", "Bash(rm -rf *)"]
  }
}
```

Edits, tests and commits run without prompts; anything that leaves the machine asks. `settings.local.json` is personal and not committed.

2.6 Optional, recommended: the Python language server

```
/plugin install pyright-lsp@claude-plugins-official
```

(`pip install pyright` first.) With it, "what type is `self.return_table`" is answered from the language server rather than from guesswork; in a 3,500-line module that pays off immediately.

Part 3 — Understand the code you are about to change

Three short exercises, each a prompt. Read the files the answers cite.

3.1 Trace the data

```
Trace what happens when I run:
    prices = pd.read_csv("tests/data/test_data.csv", index_col=0, parse_dates=True)
    s = prices["AAPL"].calc_stats()
    s.display_monthly_returns()
Walk through each function with file:line, show where return_table is computed and how a
month with no data (for example months after the last price date) is represented in it.
```

The last point matters for the feature: in the current build the months after the end date appear as **0.00**, not blank. Your issue will say what the HTML should do about that.

3.2 Study the pattern you will mirror

```
Explain to_csv() on both PerformanceStats and GroupStats line by line: how it iterates _stats(),
how blank rows and the risk-free row are handled, how values are formatted, and how the
path argument changes the return value. Then list the tests that cover to_csv.
```

3.3 Check the blast radius

```
I want to add to_html(path=None) to PerformanceStats and GroupStats, mirroring to_csv, with a
colour-coded monthly returns table. List every file that would change, whether any new
dependency is needed, and any existing helper I should reuse. Do not change anything.
```

What you should see: `ffn/core.py` (two methods plus one or two private helpers), `tests/test_core.py` (or a new `tests/test_html_report.py`), the README or docs if they list report methods; no new dependency (string formatting is enough; note that jinja2 is *not* a dependency of ffn); reuse `_stats()`, `_format_stat_value()`, `fmtpn()`, `return_table`, `_names`.

Quick side question without touching the main context:

```
/btw how many tests mention to_csv, and in which files?
```

Part 4 — Tools that help with understanding, reviewing and securing an existing codebase

Before writing the issue, it is worth knowing what is already available for the work you have just done by hand in Parts 2 and 3. The list below is restricted to what is **built into Claude Code**, what is **listed in Anthropic's official plugin marketplace** (`claude-plugins-official`, re-checked on 30 Oct 2026), and community packages whose repositories have **at least 1,000 GitHub stars**. Items marked ★ are the ones used or suggested in this demo; the rest are there for reference.

4.1 Understanding architecture, design and code

Item	Status	What it does	How to use it here
★ Plan mode + the `Explore` subagent	Built in	Read-only exploration; wide searches run in a separate context so the main one stays small	Part 2.3 and Part 3 run in plan mode
★ Mermaid diagrams from prompts	Built in	Claude writes class, sequence, dependency or flow diagrams of code it has read; they render in GitHub, VS Code and most Markdown viewers	See 4.5 below
★ `/init`, `CLAUDE.md`, `/memory`	Built in	Survey the repository; standing rules every session; durable notes	Parts 2.2, 2.4, 7.1
★ `code-modernization`	Official (Anthropic)	Built for legacy codebases: `/code-modernization:modernize-assess` (inventory, complexity, debt, security), `modernize-map` (dependencies, data flow, entry points and business flows as an **interactive map**), `modernize-extract-rules` (business rules as Given/When/Then cards with `file:line`), all gathered in one `REPORT.html`. "Understand it first" mode changes no code	`/plugin install code-modernization@claude-plugins-official`, then `/code-modernization:modernize` and choose *Understand it first*; point it at `ffn/` (see 4.5)
★ `feature-dev`	Official (Anthropic)	`/feature-dev` runs three agents in sequence: `code-explorer` (understand the codebase), `code-architect` (design), `code-reviewer` (quality); asks clarifying questions before designing	An alternative to the hand-written prompts in Part 6.1
★ `claude-md-management`	Official (Anthropic)	`/revise-claude-md`: audits `CLAUDE.md` quality and captures session learnings into it	Part 7.1

4.2 Code review

Item	Status	What it does	How to use it here
★ `/code-review`	Built in	Reviews the current diff	Part 6.3
★ `pr-review-toolkit`	Official (Anthropic)	Agents for comments, tests, error handling, type design, code quality and simplification; `/review-pr`	Part 7.4
`code-review`	Official (Anthropic)	Multi-agent pull-request review with confidence scoring that filters false positives	`/plugin install code-review@claude-plugins-official`; run on the PR from Part 6.4
`code-simplifier`	Official (Anthropic)	Agent that simplifies recently modified code while preserving behaviour	After the implementation, before the PR
★ `superpowers`	Official listing (external, obra; very widely adopted)	Process skills: brainstorming, systematic-debugging, requesting-code-review, receiving-code-review, test-driven development, subagent-driven development	`/plugin install superpowers@claude-plugins-official`; opinionated, trial in a worktree first

4.3 Security review

Item	Status	What it does	How to use it here
★ `/security-review`	Built in	Security review of the diff against the default branch	Part 6.3
★ `security-guidance`	Official (Anthropic)	Pattern-based warnings on edits, LLM diff review on Stop, an agentic commit reviewer covering injection, XSS, SSRF, hard-coded secrets and 25+ other classes	`/plugin install security-guidance@claude-plugins-official`
`claude-security`	Official (Anthropic)	Deep vulnerability scan of your own code at a chosen effort tier; every finding is challenged before it is reported; verified patches you apply when you choose	For a one-off audit of the whole repository

4.4 Issues, pull requests and hand-over

Item	Status	What it does	How to use it here
★ `gh` CLI	External (GitHub)	Issues, PRs, reviews from the shell; the model uses it too	Parts 5 and 6
`github`	Official listing (GitHub MCP server)	Issues, PRs, code review, search through MCP tools	Alternative to `gh` when you want the model to read review comments as structured data
★ `/export` and a written architecture note	Built in	Transcript export; an `ARCHITECTURE.md` produced from the briefing	Part 7.5; see 4.5

Everything in the earlier research that was community-only and below 1,000 stars (C4 and Excalidraw diagram plugins, codebase visualisers, code knowledge-graph MCP servers, adversarial review bundles) is deliberately left out; `code-modernization`'s map plus Mermaid prompts cover the same ground from official or built-in sources.

4.5 Two optional exercises with these tools (15 minutes)

Draw it (no install). In plan mode:

```
Draw a Mermaid class diagram of PerformanceStats and GroupStats in ffn/core.py: attributes that
hold data (prices, returns, return_table, _names, ...), the reporting methods, and the
module-level calc_* functions they call. Then a Mermaid sequence diagram for
prices["AAPL"].calc_stats().display_monthly_returns(). Save both to docs/ARCHITECTURE.md with
one paragraph of explanation each.
```

Open `docs/ARCHITECTURE.md` on GitHub (after the PR) or in VS Code's Markdown preview; both render Mermaid. The diagram is the mind map of the part you are about to change, and it is in the repository for the next person.

Map it (official plugin).

```
/plugin install code-modernization@claude-plugins-official
/code-modernization:modernize
```

Choose *Understand it first*, name it `ffn`, point `--source` at the repository. Run `modernize-assess`, then `modernize-map`, and open `analysis/ffn/REPORT.html`: the dependency and data-flow map of `core.py`, the entry points, and the business flows it found. Compare the map with the briefing you produced by prompt in Part 2.3; note what each found that the other missed. The plugin writes only under `analysis/` and `modernized/`; add `analysis/` to `.gitignore` (or `.git/info/exclude`).

4.6 Security review in CI on your fork (optional, 5 minutes)

After the PR in Part 6 is merged, add `.github/workflows/security-review.yml` to your fork using the `anthropics/claude-code-security-review` action with an `ANTHROPIC_API_KEY` repository secret. Every later pull request gets a security review comment automatically; it is the same check as `/security-review`, run where the team sees it.

Part 5 — Create the issue on GitHub

Everything the model does in Part 6 starts from the issue text, so the issue is the specification. Write it with acceptance criteria, not a wish.

5.1 Issue text

> **Title:** Add `to_html()` report with colour-coded monthly returns
>
> **Problem.** `PerformanceStats` and `GroupStats` can print to the terminal (`display*`), write CSV (`to_csv`) and draw charts (`plot*`), but there is no way to produce a shareable report. Analysts paste terminal output into emails.
>
> **Proposal.** Add `to_html(path=None, title=None)` to both classes.
>
> **Acceptance criteria**
>
> 1. `PerformanceStats.to_html()` returns a complete, self-contained HTML document (inline CSS, no external assets) containing: a heading with the strategy name and date range; the summary table built from the same `_stats()` rows and formatting as `to_csv`; the monthly returns table (`return_table`: years as rows, Jan–Dec and YTD as columns) with values in percent to two decimals.
> 2. Monthly return cells are colour-coded: positive values carry CSS class `pos` and a green background, negative values class `neg` and a red background, shaded by magnitude (darker for larger absolute returns, capped at ±10%). Zero values have no colour.
> 3. Months with no data are rendered blank (`-`), not as `0.00`. (Note: `return_table` currently holds 0.0 for months after the series end; the HTML must detect them from the series' end date.)
> 4. `GroupStats.to_html()` renders one section per strategy in `_names` order, separated by a rule, in a single document.
> 5. With `path` given, the file is written (UTF-8) and the method returns `None`; otherwise the string is returned. Same contract as `to_csv`.
> 6. Strategy names are rendered with `str(name)` and HTML-escaped; a numeric strategy name works.
> 7. No new dependencies.
> 8. Tests for both classes using `tests/data/test_data.csv` only (no network): document structure, `pos` / `neg` classes on known months, blank cells after the end date, file written with `path`, numeric name.
> 9. README (or docs) mentions the new method next to `to_csv`.
>
> **Out of scope:** charts in the HTML, Jinja templates, styling options.

5.2 Create it

From the web: github.com/<you>/ffn → **Issues** → **New issue** → paste title and body → **Submit**.

From the terminal (the text above saved as `issue.md`):

```
gh issue create --repo <you>/ffn --title "Add to_html() report with colour-coded
monthly returns" --body-file issue.md
gh issue list --repo <you>/ffn
```

From inside Claude Code (shell mode):

```
!gh issue list --state open
```

Note the issue number; it is `#1` on a fresh fork. If `gh issue create` says issues are disabled, enable them in the fork's **Settings → General → Features → Issues** (forks have issues off by default).

5.3 What makes an issue good for a model

- Criteria that a test can check (classes `pos` / `neg`, blank cells, return value by `path`).
- The constraints stated (no dependencies, mirror `to_csv`, offline tests).
- The known trap named (the `0.00` months), so it is a requirement, not a surprise.
- Out-of-scope listed, so the model does not add charts "while it is there".

Part 6 — Let Claude Code fix the issue and open the pull request

6.1 Fresh context, then the issue

```
/clear
```

```
Work on GitHub issue 1 in this repository (gh issue view 1).
Before changing any code:
  a. Restate the acceptance criteria as a checklist.
  b. Create branch feature/html-report from master.
  c. Write the tests first (tests/test_html_report.py) using tests/data/test_data.csv; run them
     and show that they fail for the right reason (AttributeError: no to_html).
Then implement to_html on PerformanceStats and GroupStats in ffn/core.py, mirroring to_csv and
reusing _stats(), _format_stat_value and fmtpn. Keep helpers private. No new dependencies.
Run the new tests, then the full suite (the float16 failure is known and expected), then lint.
Update the README next to to_csv. Commit in two commits: tests, then implementation. Do not push.
```

Watch for:

- The failing-test step. If it skips to implementation, stop it: "tests first, show them failing."
- Where the blank-month rule is implemented. The honest solution compares each (year, month) against `self.end`; a hack that treats every `0.0` as blank will also blank a month that returned exactly zero. Ask: "how do you distinguish a genuine 0.00 month from a month after the end date?"
- Whether it touched anything outside the issue. `!git diff master --stat` should show `ffn/core.py`, one test file, the README.

6.2 Look at the output

```
Write a report for AAPL from tests/data/test_data.csv to /tmp/aapl.html using the new method,
and a GroupStats report for all three columns to /tmp/all.html.
```

Open the files in a browser (macOS `open /tmp/aapl.html`; PowerShell `Start-Process C:\path\aapl.html`).
What you should see: the summary table, then the monthly grid with 2008 mostly red, 2009 mostly green, and the months of 2014 after March shown as `-`.

6.3 Review before it leaves the machine

```
/code-review
```

Then your own questions:

```
Stash the implementation and run tests/test_html_report.py to prove the tests fail without it.
Then restore it.
```

```
Run the slice of the suite that covers to_csv and display* to prove nothing shared was changed.
```

For a feature that writes files, `/security-review` is cheap insurance (path handling, escaping of names).

Optionally, a second opinion from a subagent that has not seen the conversation: paste the `change-reviewer` definition from the existing-project guide into `.claude/agents/change-reviewer.md` and ask it to review the branch.

6.4 Push and open the pull request against your fork's `master`

```
Push feature/html-report to origin and open a pull request against master on <you>/ffn with
gh pr create --repo <you>/ffn --base master. Title under 70 characters. Body: what was missing,
what was added, how the blank-month rule works, how it was tested, screenshots omitted,
"Closes #1".
```

The push and the `gh pr` command both stop at your **ask** permission; approve each. If `gh pr create` offers `pmorissette/ffn` as the base, answer with your fork (this is what step 1.3 prevents).

What you should see: a PR URL on `github.com/<you>/ffn/pull/1`, linked to issue #1.

6.5 Merge and verify

On GitHub: read the diff and the tests once more, then **Merge pull request** (squash or merge, your choice). Issue #1 closes automatically.

```
!git checkout master
!git pull
!python -m pytest -q -p no:cacheprovider tests/test_html_report.py
```

Regenerate the report from `master` and open it: the feature is now in your fork.

Part 7 — Steps that add value beyond the feature

7.1 Record what you learned

```
Add to CLAUDE.md: return_table contains 0.0 for months after the series end; report code must
use self.end to blank them. Keep it to one line under conventions.
```

Facts like this are what the next contributor (or the next session) does not know.

7.2 A hook that runs the report tests after every edit

`.claude/settings.local.json`, under `"hooks"`:

```
"PostToolUse": [{ "matcher": "Edit|Write",
  "hooks": [{ "type": "command",
    "command": "python -m pytest -q -x -p no:cacheprovider
tests/test_html_report.py 2>&1 | tail -2" }] }]
```

(Windows runs this through Git Bash.) Fast, targeted, and the result lands in the context after each change.

7.3 Try a second task in a worktree

```
claude --worktree group-colours
```

A second issue to practise with, on an isolated checkout: *"GroupStats.to_html should add a comparison table (one row per strategy, total return / CAGR / max drawdown / Sharpe) above the per-strategy sections."* Same loop: issue → failing test → implement → review → PR.

7.4 Compare with the model's own review tooling

```
/plugin install pr-review-toolkit@claude-plugins-official
/review-pr 1
```

Compare its findings with `/code-review` and with your own reading. The point is not which is best; it is that reviews from different contexts catch different things.

7.5 Export the session

```
/export ffn-html-report-session.txt
```

Keep it with the repository (outside Git or under `docs/`). It is the record of the decisions, and good material for a retrospective: where did the model need correcting, and what in the issue text prevented a wrong turn?

7.6 Offer it upstream, if you want to

```
gh pr create --repo pmorissette/ffn --base master --head <you>:feature/html-report
```

Upstream's default branch is also `master`; read the README's contribution notes first. Expect review comments; Part 5.3 of the existing-project guide covers responding to them with Claude Code.

Part 8 — Troubleshooting

Symptom	Fix
`gh repo fork` says the fork already exists	`gh repo clone <you>/ffn && cd ffn && git remote add upstream https://github.com/pmorissette/ffn.git`
`gh issue create` fails with "issues are disabled"	Enable Issues in the fork's Settings → General → Features
`gh pr create` targets `pmorissette/ffn`	`gh repo set-default <you>/ffn`, or pass `--repo <you>/ffn --base master`
Claude Code tries to fix the float16 test	It was not told; add the known-failure line to `CLAUDE.md` and `/clear`
Tests call the network (`ffn.get`)	Point to `tests/data/test_data.csv`; add "no network in tests" to `CLAUDE.md`
`pip install -e .` is slow	matplotlib and scipy are dependencies; wait, or pre-install them before the session
HTML shows `0.00` for future months	Criterion 3 was not met; ask how a genuine zero month is distinguished from a missing one
Push asks for a password	`gh auth setup-git`

Appendix A — Reference implementation (instructor copy)

A working version, verified against `tests/data/test_data.csv`, is in `ffn_ref/html_report.py` (shipped with this guide). It implements the two methods as module-level functions so it can be run without modifying the repository:

```
python ffn_ref/html_report.py aapl_report.html     # writes aapl_report.html and aapl_report_all.html
```

In the demo the same logic becomes `PerformanceStats.to_html` and `GroupStats.to_html` inside `ffn/core.py`. Note that the reference deliberately does **not** implement criterion 3 (blank months after the end date): it renders `return_table` as is, so trailing months show `0.00`. That is the part participants must think about; a correct implementation compares `(year, month)` with `self.end`.

Appendix B — Reference tests (instructor copy)

```
import pandas as pd
import pytest
import ffn  # noqa: F401  registers .calc_stats


@pytest.fixture(scope="module")
def prices():
    return pd.read_csv("tests/data/test_data.csv", index_col=0, parse_dates=True)


def test_perf_to_html_structure(prices):
    html = prices["AAPL"].calc_stats().to_html()
    assert html.startswith("<!DOCTYPE html>")
    assert "Total Return" in html and "Monthly returns" in html
    assert "<link" not in html and "<script" not in html        # self-contained


def test_perf_to_html_colours(prices):
    html = prices["AAPL"].calc_stats().to_html()
    assert 'class="neg"' in html and 'class="pos"' in html
    # 2008 Jan was -31.66%; 2009 Mar was +17.70%
    assert '<td class="neg" style="background:rgba(178,34,34' in html


def test_perf_to_html_blank_after_end(prices):
    s = prices["AAPL"].calc_stats()           # series ends 2014-03-10
    html = s.to_html()
    row_2014 = html.split("<td>2014</td>", 1)[1].split("</tr>", 1)[0]
    assert row_2014.count('class="blank"') == 9     # Apr..Dec blank


def test_group_to_html_sections_and_path(prices, tmp_path):
    g = prices.calc_stats()
    html = g.to_html()
    assert html.count("<h1>") == 3 and html.index("AAPL") < html.index("MSFT") <
html.index(">C<")
    out = tmp_path / "r.html"
    assert g.to_html(path=out) is None
    assert out.read_text(encoding="utf-8") == html


def test_numeric_name(prices):
    s = prices["AAPL"].rename(42).calc_stats()
    assert "42" in s.to_html()
```

Appendix C — Session script (90 minutes)

Minutes	Step
0–10	Fork, clone, `set-default`, baseline run; note the known failure
10–25	`/init`, plan-mode briefing, `CLAUDE.md`, permissions, `/context all`
25–35	Trace `return_table`; study `to_csv`; blast radius
35–40	Part 4: the tool map; optionally draw the Mermaid diagram
40–45	Write and create the issue; discuss what makes it testable
45–70	Claude Code: failing tests → implementation → suite → README → commits
70–80	Browser check; `/code-review`; stash-and-prove; push; PR; merge
80–90	Record the lesson in `CLAUDE.md`; export; discuss the worktree follow-up